

CSR Format:

- ↳ Stores all out-going edges for a node
- ↳ Only vertices who are assigned a level check their neighbours

CSC Format

- ↳ Stores all in-coming edges for a node
- ↳ Only unassigned vertices check their src nodes.

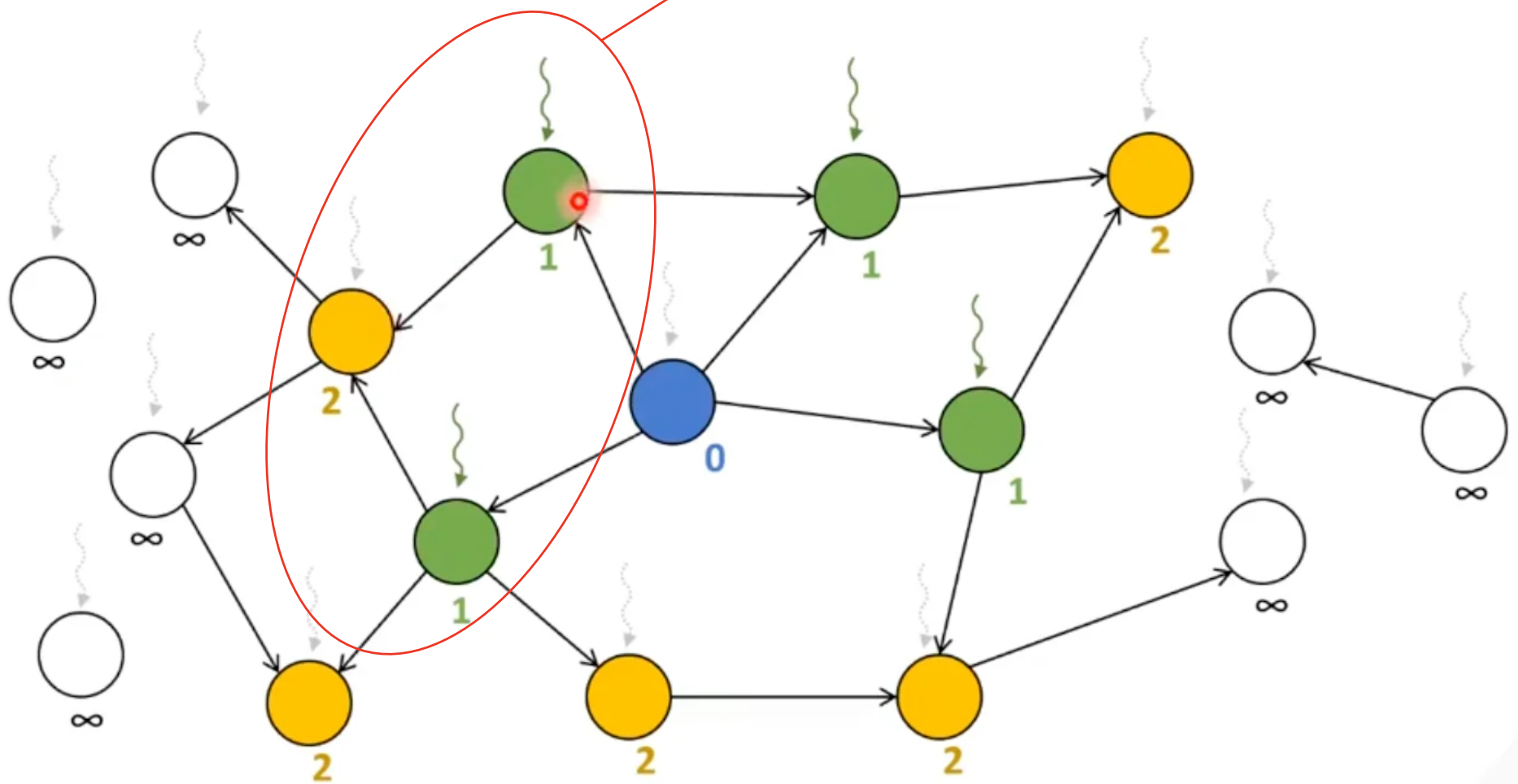
COO Format

- ↳ Stores edges

Push Implementation

```
01 __global__ void bfs_kernel(CSRGraph csrGraph, unsigned int* level,  
02 unsigned int* newVertexVisited, unsigned int currLevel) {  
03     unsigned int vertex = blockIdx.x*blockDim.x + threadIdx.x;  
04     if(vertex < csrGraph.numVertices) {  
05         if(level[vertex] == currLevel - 1) {  
06             for(unsigned int edge = csrGraph.srcPtrs[vertex];  
07                 edge < csrGraph.srcPtrs[vertex + 1]; ++edge) {  
08                 unsigned int neighbor = csrGraph.dst[edge];  
09                 if(level[neighbor] == UINT_MAX) { // Neighbor not visited  
10                     level[neighbor] = currLevel;  
11                     *newVertexVisited = 1;  
12                 }  
13             }  
14         }  
15     }  
16 }
```

Idempotent race condition but in cpp semantics, this results in undefined behaviour



Level 1 => Level 2

- The best parallelization approach depends on the structure of the graph
- The vertex-centric pull and the edge-centric approaches are better on high-degree graphs
 - e.g., social network graphs with celebrities
 - Better at dealing with load imbalance
- The vertex-centric push approach is better on low-degree graphs
 - e.g., map of roads in a geographical area
 - Only promising threads will iterate through neighbors
 - The levels progress slowly so push approach works better as only threads' vertices who have already been assigned a level run i.e. for large num iteration there are more unassigned nodes than assigned.

Privatization

```

01 __global__ void bfs_kernel(CSRGraph csrGraph, unsigned int* level,
02     unsigned int* prevFrontier, unsigned int* currFrontier,
03     unsigned int numPrevFrontier, unsigned int* numCurrFrontier,
04     unsigned int currLevel) {
05
06     // Initialize privatized frontier
07     __shared__ unsigned int currFrontier_s[LOCAL_FRONTIER_CAPACITY];
08     __shared__ unsigned int numCurrFrontier_s;
09     if(threadIdx.x == 0) {
10         numCurrFrontier_s = 0;
11     }
12     __syncthreads();
13
14     // Perform BFS
15     unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
16     if(i < numPrevFrontier) {
17         unsigned int vertex = prevFrontier[i];
18         for(unsigned int edge = csrGraph.srcPtrs[vertex];
19             edge < csrGraph.srcPtrs[vertex + 1]; ++edge) {
20             unsigned int neighbor = csrGraph.dst[edge];
21             if(atomicCAS(&level[neighbor], UINT_MAX, currLevel) == UINT_MAX) {
22                 unsigned int currFrontierIdx_s = atomicAdd(&numCurrFrontier_s, 1);
23                 if(currFrontierIdx_s < LOCAL_FRONTIER_CAPACITY) {
24                     currFrontier_s[currFrontierIdx_s] = neighbor;
25                 } else {
26                     numCurrFrontier_s = LOCAL_FRONTIER_CAPACITY;
27                     unsigned int currFrontierIdx = atomicAdd(numCurrFrontier, 1);
28                     currFrontier[currFrontierIdx] = neighbor;
29                 }
30             }
31         }
32     }
33     __syncthreads();
34
35     // Allocate in global frontier (Reserve numCurrFrontier_s space)
36     __shared__ unsigned int currFrontierStartIdx;
37     if(threadIdx.x == 0) {
38         currFrontierStartIdx = atomicAdd(numCurrFrontier, numCurrFrontier_s);
39     } // start pos of reserved space
40     __syncthreads();

```

```

41     __syncthreads();
42     // Commit to global frontier
43     for(unsigned int currFrontierIdx_s = threadIdx.x;
44         currFrontierIdx_s < numCurrFrontier_s; currFrontierIdx_s += blockDim.x) {
45         unsigned int currFrontierIdx = currFrontierStartIdx + currFrontierIdx_s;
46         currFrontier[currFrontierIdx] = currFrontier_s[currFrontierIdx_s];
47     }
48
49 }

```

Kernel Launch Overhead

```

__global__ void bfs_kernel(CSRGraph csrGraph, unsigned int* level, unsigned int* prevFrontier, unsigned int* numCurrFrontier) {
    grid_group grid = this_grid();

    for(unsigned int currLevel = 1; numPrevFrontier > 0; ++currLevel) {
        // Initialize private frontier
        __shared__ unsigned int currFrontier_s[PRIVATE_FRONTIER_CAPACITY];
        __shared__ unsigned int numCurrFrontier_s;
        if(threadIdx.x == 0) {
            numCurrFrontier_s = 0;
        }
        __syncthreads();

```

because coop
gpus limits threads
→ launched

```

// Perform BFS
for(unsigned int i = blockIdx.x*blockDim.x + threadIdx.x; i < numPrevFrontier; i += blockDim.x*gridDim.x) {
    unsigned int vertex = prevFrontier[i];
    for(unsigned int edge = csrGraph.srcPtrs[vertex]; edge < csrGraph.srcPtrs[vertex + 1]; ++edge) {
        unsigned int neighbor = csrGraph.dst[edge];
        if(visitVertexAtomically(neighbor, level, currLevel)) {
            cuda::atomic_ref<unsigned int, cuda::thread_scope_block> numCurrFrontier_s_ref(numCurrFrontier_s);
            unsigned int currFrontierIdx_s = numCurrFrontier_s_ref.fetch_add(1, cuda::memory_order_relaxed);
            if(currFrontierIdx_s < PRIVATE_FRONTIER_CAPACITY) {
                currFrontier_s[currFrontierIdx_s] = neighbor;
            } else {
                numCurrFrontier_s = PRIVATE_FRONTIER_CAPACITY;
                cuda::atomic_ref<unsigned int, cuda::thread_scope_device> numCurrFrontier_ref(numCurrFrontier);
                unsigned int currFrontierIdx = numCurrFrontier_ref.fetch_add(1, cuda::memory_order_relaxed);
                currFrontier[currFrontierIdx] = neighbor;
            }
        }
    }
}
__syncthreads();

```

```

// Commit to public frontier
for(unsigned int currFrontierIdx_s = threadIdx.x; currFrontierIdx_s < numCurrFrontier_s; ++currFrontierIdx_s) {
    unsigned int currFrontierIdx = currFrontierStartIdx + currFrontierIdx_s;
    currFrontier[currFrontierIdx] = currFrontier_s[currFrontierIdx_s];
}

// Synchronize
grid.sync();
numPrevFrontier = *numCurrFrontier;
grid.sync(); → WAR dependencies
if(grid.thread_rank() == 0) {
    *numCurrFrontier = 0;
}
grid.sync();
unsigned int* tmp = prevFrontier;
prevFrontier = currFrontier;
currFrontier = tmp;
}

```

swap curr & prev frontier
for next level